

TRABALHO FINAL

Crie um jogo de luta de robôs em Python utilizando os conceitos de orientação a objetos.

Comece com uma classe base `Robo`, com as seguintes propriedades:

- Atributos de instância: `nome: str`, `vida: float`
- Métodos: `__init__`, `__repr__`, `__add__`, `precisa_de_medico`
- A classe de um Robô deve definir, para todos os robôs existentes, um valor denominado `nivel_critico`, cujo valor deve ser escolhido pelo programador (ex: `0.60`).
- Um robô “ganha vida” com um valor do atributo `vida` definido aleatoriamente entre `0` e `1`. Portanto, a vida não deve ser passada como parâmetro do construtor.
- O nome do robô deve ser passado no construtor, durante a instanciação do objeto.
- Um robô é capaz de se reproduzir com outro robô. A reprodução deve ser chamada pelo operador `+` a partir de um objeto robô (utilizar sobrecarga de operador). O nome do robô “bebê” consistirá na concatenação dos nomes de ambos os pais separados por um hífen. Se um pai tiver um nome contendo um hífen, deve ser considerada apenas a primeira parte, antes do hífen.
- O método `precisa_de_medico` deve verificar o nível de vida do robô e retornar verdadeiro caso seja menor que o nível crítico, ou falso, caso contrário.

Crie outra classe de robôs, os robôs-médicos, definidos pela classe `RoboMedico`, que deve ser subclasse de `Robo`, com as seguintes características:

- Um robô-médico pode curar outro robô caso a vida dele seja menor que `1`. Logo, deve haver um método nesta classe denominado `curar`, que recebe como parâmetro o objeto robô que será curado.
- Para que um robô-médico seja capaz de curar, o seu nível da vida deve maior ou igual ao nível de vida do robô a ser curado.
- Deve haver um atributo de instância adicional denominado `poder_de_cura: float`, que deve ser definido automaticamente, no construtor, como um número aleatório entre `0` e `1`.

Uma outra classe de robôs são os robôs-lutadores, definidos na classe `RoboLutador`, subclasse de `Robo`, com as seguintes características:

- Todos os robôs-lutadores compartilham um mesmo atributo de classe denominado `dano_maximo`, definido pelo programador (ex: `0.2`)
- Deve haver um parâmetro que representa a força, ou poder de luta do robô, denominado `poder`. O poder deve ser calculado na instanciação do robô como sendo um valor aleatório entre `dano_maximo` da classe e `1`.
- O robô-lutador tem um método denominado `atacar`, que recebe como parâmetro um outro objeto robô com o qual irá lutar. No ataque, o atributo `vida` do robô atacado deve ser multiplicado por $1 - \text{poder}$ do robô atacante.
- Durante o ataque, se o robô atacado também for um robô-lutador, ele irá contra-atacar. Portanto, a regra de redução de vida anterior será aplicada novamente, mas invertendo os papéis de atacante e atacado.

Agora, escreva um código de exemplo que instancie e teste vários robôs e os faça lutar e curar entre si:

- No decorrer das lutas, caso a vida de um robô seja menor que `0.1`, o robô deverá chamar um robô-médico. O médico poderá ou não atender a um chamado aleatoriamente, com uma probabilidade de 50%.
- Utilize listas em Python para armazenar uma série de robôs de um determinado tipo. Por exemplo, uma lista para robôs, outra para robôs-lutadores e outra para robôs-médicos.

Sugestões gerais:

- Utilize propriedades através de *decorators* para definir *getters* e *setters* quando necessário.
- Organize seus arquivos de código-fonte da maneira que julgar mais apropriada, mas procure, ao menos, definir cada classe em um arquivo separado.
- A vida dos robôs nunca deve ser maior que `1` ou menor que `0`.
- Caso um robô de uma das subclasses (`RoboMedico` ou `RoboLutador`) se reproduza com outro robô da classe base `Robo`, seu filho também deve ser da classe específica. Mas isso não ocorre por padrão. Para resolver essa situação, você deve retornar, na sobrecarga do método `__add__`, uma instância dinamicamente de acordo com o tipo atual. Exemplo:

```
return type(self)(novo_nome)
```

Referências importantes:

- Geração de números aleatórios: <https://docs.python.org/pt-br/3/library/random.html>
- Split de *strings*: <https://docs.python.org/pt-br/3/library/stdtypes.html#str.split>